

UNIT 4 – Data Visualization Techniques

Activity 2: Visualize the data appropriately using different tools and techniques.

Task 1: Dataset Selection and Preparation

- Select one medium to large dataset
- Load the same dataset into Matplotlib, Seaborn, and Tableau
- Ensure consistent data cleaning and structure

Dataset Selection :- A **medium to large dataset** was selected to ensure meaningful visualization and analysis.

Chosen Dataset: Crime Dataset (US Crime / City Crime Records)

- Size: ~100,000+ records
- Attributes: Crime type, location, date, time, victim details, weapon used, etc.
- Data Types: Numerical, categorical, temporal
- Suitable for statistical and visual analysis

Reason for Selection:

- Real-world relevance
- Mixed attribute types
- Supports trend, comparison, and distribution analysis
- Works well across Matplotlib, Seaborn.

Data Cleaning Process (Uniform for All Platforms)

To maintain consistency, the same cleaned dataset is used in Matplotlib, Seaborn

Task 2: Visualization Construction Across Tools

- Create at least three identical chart types (bar, line, scatter, distribution)
- Maintain identical axes, scales, and filters
- Capture outputs for comparison

Task 3: Interactivity and Customization Testing

- Apply filters, hover details, or interactions
- Customize colors, labels, legends, and layouts
- Consider clarity and accessibility

Chart 1: Bar Chart :- Crime Count by Crime Type

Axes (Fixed)

- X-axis: Crime Type
- Y-axis: Number of Crimes
- Scale: Count (no normalization)

Chart 2: Line Chart :- Monthly Crime Trend

Axes (Fixed)

- X-axis: Month (1–12)
- Y-axis: Crime Count
- Aggregation: COUNT

Chart 3: Scatter Plot :- Victim Age vs Crime Occurrence Time

Axes (Fixed)

- X-axis: Occ Hour
- Y-axis: Vict Age
- Scale: Linear

- Filter: Vict Age $\leq$ 80

Chart 4: Distribution Plot :- Victim Age Distribution

Axes (Fixed)

- X-axis: Vict Age
- Y-axis: Frequency
- Bins: 10-year ranges

Task 4: Performance Benchmarking and Observation

- Measure rendering time
- Observe responsiveness during interactions
- Record scalability behavior

Rendering Time Measurement:-

- Matplotlib Rendering Time :- 89.95753359794617 seconds
- Seaborn Rendering Time :- 104.06750392913818 seconds

Responsiveness Observations

Aspect	Matplotlib	Seaborn
Filter change	Fast	Slightly Slower
Redraw speed	High	Moderate
Visual update	Immediate	Slight delay
Smoothness	High	Acceptable

Scalability Observations

Dataset Size	Matplotlib	Seaborn
5k rows	Very Fast	Fast
20k rows	Fast	Moderate
50k rows	Slight lag	Noticeable lag
100k+ rows	Heavy	Slower

Task 5: Comparative Evaluation and Documentation

- Prepare comparison tables
- Evaluate strengths, limitations, and use cases
- Document findings with evidence

Performance Comparison:-

Aspect	Matplotlib	Seaborn
Rendering Time	Faster	Slightly lower
Memory Usage	Lower	Higher
Large Dataset Handling	Better	Moderate
Redraw Responsiveness	Immediate	Slight lag

Strengths and Limitations:-

Matplotlib:-

Strengths

- High performance
- Full control over plot elements
- Better scalability
- Suitable for production-level plots

Limitations

- More code required
- Less attractive defaults
- Steeper learning curve

Seaborn:-

Strengths

- Attractive visuals with minimal code
- Strong statistical plotting support

- Better default accessibility
- Ideal for exploratory analysis

Limitations

- Slower for large datasets
- Less low-level control
- Dependent on Matplotlib

Use-Case Suitability:-

Use Case	Recommended Tool
Large datasets	Matplotlib
Statistical analysis	Seaborn
Quick EDA	Seaborn
Publication-ready plots	Matplotlib
Teaching & learning	Seaborn
Performance-critical apps	Matplotlib